Graphic Processing

Technical University of Cluj-Napoca

Final project - Interactive Scene Using Modern OpenGL

Student: Daria Nicola Colopelnic

Group: 30432

Year of study: 2025 – 2026

Contents

1 Subject specification

This project develops a real-time 3D interactive scene using modern OpenGL. The main objective is to design and implement a dynamic environment that demonstrates the fundamental concepts of computer graphics, using a modular and extensible architecture developed in C++.

The application allows the user to explore a virtual outdoor environment using keyboard and mouse controls, interact with both static and animated objects placed within the scene, and trigger different features through user input.

To enhance visual realism and immersion, the project integrates multiple graphical effects such as shadow mapping, fog, wind animation, and multiple light sources.

The implementation incorporates the following features:

- Camera movements using keyboard and mouse input
- Object transformations (translation, rotation, scaling)
- Different types of light sources (directional and point lights)
- Textured and detailed 3D models
- Shadow computation
- Object animation
- Collision detection

2 Scenario

2.1 Scene and objects description

The implemented application represents a scene simulating a small countryside setting containing natural elements, buildings, animals, and functional objects, arranged to create a coherent and realistic environment.

The two central elements of the scene are a house and a fenced area in which different animal models were placed. The wooden fence includes an animated gate that can be opened and closed by user input, establishing an area that supports interaction and collision detection.

The environment includes multiple natural elements, such as a ground surface textured to resemble terrain, illustrating small hills and variations in relief, and multiple trees, distributed along the edges of the scene to frame the environment. The trees are animated with a wind effect, simulating natural movement and adding to the realism of the scene.

Several animal models are placed at different positions and orientations, including domestic and farm animals. Some of these objects include simple rotation animations, to avoid static visuals and highlight the use of lighting and shadow computation.

Additionally, two lamp objects are placed in the scene and act as point light sources. These lamps provide localized lighting and contribute to the overall atmosphere of the environment.

The scene is surrounded by a skybox, which provides a distant background and enhances immersion by simulating a realistic sky and horizon.

Overall, the project creates a cozy rural scene which balances visual complexity and performance, while demonstrating multiple rendering techniques.

2.2 Functionalities

The application provides multiple interactive and graphical functionalities that allow the user to explore and interact with the scene in real time.

Camera Control and Navigation:

- The user can move freely through the scene using the keyboard.
- Mouse input allows camera rotation, in a first-person viewing.
- Movement is constrained by collision detection to prevent passing through solid objects such as fences and trees.

Object Interaction:

- The gate can be opened and closed using keyboard input (key G), triggering a smooth animation.
- Animals can be rotated using keyboard input (keys Q and E).

Lighting and Visual Effects:

- Fog can be toggled on and off (key F), to enhance depth perception.
- Wireframe rendering mode can be enabled, allowing visualization of polygonal versus smooth surfaces.

Environmental Effects:

- A wind effect is applied to trees, creating subtle vertex displacement, simulating natural movement.

3 Implementation details

This chapter describes the core implementation aspects of the application, focusing on the algorithms, data structures, and the programmable graphics pipeline used to render the 3D environment.

3.1 Functions and special algorithms

The application integrates several core algorithms that support interaction and realism including camera movement and collision detection, shadow mapping, object animation, and environmental effects such as wind and fog.

Camera Movement and Collision Detection

The camera is implemented using a first-person navigation model, where the view matrix is updated based on user input and mouse delta. Collision detection is handled using a system that combines:

- Radius-based Collisions: Used for trees, where a 2D distance check on the XZ plane determines if the camera has entered a tree's proximity.
- Oriented Bounding Boxes (OBB): Used for fence segments and the gate. The system performs an axis-aligned check to prevent passing through objects.

The collision system prevents the camera from passing through solid objects by reverting movement when an intersection is detected, ensuring a natural navigation experience.

Shadow Mapping

To produce realistic lighting, a Shadow Mapping algorithm is implemented using a two-pass rendering technique:

1. Depth Pass: The scene is first rendered from the perspective of the directional light source using an Orthographic Projection. The resulting depth values are stored in a Framebuffer Object (FBO) as a depth texture (Shadow Map).
2. Lighting Pass: During final rendering, each fragment’s world position is transformed into light-space. The fragment's depth is compared against the sampled value from the Shadow Map to determine if the point is in shadow or not.

Object Animation

Object animations, such as animal rotation and gate movement, are implemented using time-based transformations on the CPU. This ensures smooth and frame-rate-independent animation. For the tree movement, vertex displacement in the vertex shader is used, using a sinusoidal function driven by time and wind direction, creating subtle and natural motion.

Environmental Effects

Atmospheric perspective is achieved through a distance-based exponential fog algorithm. Fog is implemented as a distance-based effect in the fragment shaders, blending scene colors with a predefined fog color based on depth.

3.2. Graphics model

The graphics model follows the modern OpenGL programmable pipeline. Rendering is performed using vertex and fragment shaders, with transformation matrices controlling object positioning and camera projection.

The rendering process consists of three main passes:

1. Shadow Map Generation, rendering the scene from the light's perspective
2. Main Geometry Pass, rendering all models using the basic and tree shaders, applying Blinn-Phong lighting, point lights, shadows, and fog.
3. Skybox Pass, rendering a cube-map as the background which appears to be at an infinite distance.

Lighting is calculated using the Blinn-Phong reflection model, combining a global directional light (sun/moon) and multiple local point lights (streetlamps) with attenuation.

3.3. Data structures

Efficiency is maintained by using structured data to manage scene complexity:

- TreeInstance and FenceSegment: structs that store the transformation matrices (position, rotation, scale) and specific data for each object.
- Collider: a simplified data structure storing a center point and a radius/half-width, used to optimize the collision detection loop.
- Standard Containers and vectors: used to manage collections, allowing for dynamic and simplified scene generation.

These structures allow flexible scene composition and simplify collision detection and rendering loops.

3.4. Class hierarchy

The application is structured using a modular class-based design. The main components include:

- Window: Manages window creation and OpenGL context
- Shader: Handles shader compilation, linking, and uniform management

- Camera: Manages camera position, orientation, and movement
- Model3D & Mesh: load complex polygonal meshes, textures, and material properties
- SkyBox: Manages the specialized cubemap texture and the geometry required for the environmental background.
- Main: Includes the main rendering loop, initialization, update logic and the main program flow.

This separation of responsibilities improves code readability, maintainability, and extensibility, allowing new features or objects to be added with minimal changes to existing code.

4 Graphical user interface presentation

The application provides a minimalistic graphical user interface, focusing on keyboard and mouse interaction.

User Controls:

- W / A / S / D – Move the camera forward, left, backward, and right
- Mouse movement – Rotate the camera
- ESC – Exit the application
- G – Open or close the gate
- Q / E – Rotate the animal model
- F – Toggle fog effect on and off
- 1 – Enable solid rendering mode
- 2 – Enable wireframe rendering mode

The interface is intuitive, allowing the user to immediately explore and interact with the scene.

5 Conclusions and further developments

This project successfully demonstrates the implementation of a real-time 3D interactive scene using modern OpenGL techniques. Core concepts of computer graphics, such as transformations, lighting models, texture mapping, shadow computation, and animation, are integrated into a coherent and visually engaging environment.

The modular structure of the application, combined with the use of shaders and structured data management, ensures both efficiency and code maintainability. Interactive features such as camera

navigation, object animation, collision detection, and environmental effects contribute to a realistic and immersive experience.

Several enhancements could be implemented in future versions of the application, including more advanced animation techniques, a system for effects such as rain, or snow, a dynamic day-night cycle with changing light conditions, a menu interface for toggling settings and effects.

These improvements would further enhance the interactivity and overall complexity of the application, resulting in a more visually engaging and elaborate scene.

5 References

[1] “Model loading”

https://learnopengl.com/Model-Loading/Model

[2] “Shadow mapping”

https://learnopengl.com/Advanced-Lighting/Shadows/Shadow-Mapping

[3] “Collision detection”

https://learnopengl.com/In-Practice/2D-Game/Collisions/Collision-detection

[4] “Collision detection”

https://www.videotutorialsrock.com/opengl_tutorial/collision_detection/text.php

[5] “Collision detection: Picking with custom Ray-OBB function”

https://www.opengl-tutorial.org/miscellaneous/clicking-on-objects/picking-with-custom-ray-obb-function/

[6] “Blinn-Phong shading”

https://glium.github.io/glium/book/tuto-13-phong.html